

Current State: Mamba-Based Residual Memory on Frozen GraphCast

Lianghong Mo

April 22, 2026

Abstract

This note summarises the architecture that is running in the `Weather_Global_lagrangian_lab` repository as of 2026-04-22: a frozen GraphCast provides a one-step Markov prediction; a small Mamba-style selective state-space network, applied independently at every grid point, learns a residual memory correction. The note covers (i) how Mamba is inserted, (ii) how training samples are constructed from contiguous time segments, and (iii) the full parameter inventory of the currently running pilot.

Contents

1	Overview	1
2	How Mamba is integrated with GraphCast	2
2.1	GraphCast skeleton	2
2.2	Insertion point: <code>mesh_post_encoder_residual</code>	2
2.3	MZ-lite variant used in the current pilot	2
2.4	Anatomy of one Mamba block	3
3	Sampling: how a training example is built	3
3.1	Year layout and time continuity	3
3.2	Sequential segments	3
3.3	One training step	4
3.4	Teacher forcing and its caveat	4
4	Loss: GraphCast-style weighted MSE in normalised units	4
5	Current parameters	5
5.1	Run configuration for the 2026-04-21 pilot	5
5.2	Resolution sweep in progress	5
6	Representative results (res=4, step 400)	6
7	Known limitations	6
8	Planned fix: from teacher forcing to autoregressive residual rollout	6
9	File layout for this pilot	8

1 Overview

We treat GraphCast as the Markov part of the dynamics. A separate, much smaller Mamba module is trained *on top of* a frozen GraphCast checkpoint and learns only the time-history-dependent residual. This splits the problem into two clean pieces:

- GraphCast (frozen): captures the instantaneous one-step physics.
- Mamba residual memory (trained): captures everything that persists across steps — the Mori–Zwanzig memory kernel, in effect.

The Mori–Zwanzig formalism writes the exact dynamics of a projected set of variables $\mathbf{q}$ as

$$\dot{\mathbf{q}}(t) = \underbrace{\mathcal{M}(\mathbf{q}(t))}_{\text{Markov}} + \underbrace{\int_0^t K(t-s) \mathbf{q}(s) ds}_{\text{memory}} + \underbrace{\eta(t)}_{\text{noise}}.$$

Our identification:

- $\mathcal{M}$: GraphCast one-step forecast,
- K : per-grid-point Mamba SSM,
- η : absorbed by MSE loss (Gaussian noise).

2 How Mamba is integrated with GraphCast

2.1 GraphCast skeleton

GraphCast follows an encode–process–decode pattern on an icosahedral mesh:

1. **Grid-to-Mesh GNN** (encoder) lifts lat/lon grid features to mesh node features of width D (`latent_size`).
2. **Mesh Processor GNN** runs message passing on the mesh (one or more rounds).
3. **Mesh-to-Grid GNN** projects mesh features back to grid predictions.

We do not modify this pipeline. GraphCast is loaded from a pre-trained checkpoint and its parameters are *frozen* for the entire training of the memory module.

2.2 Insertion point: `mesh_post_encoder_residual`

Within the main GraphCast/Mamba codebase (`Weather_Global_experiments`), Mamba can be inserted at one of several locations. For the residual-memory setup we use `mesh_post_encoder_residual`: after the Grid-to-Mesh encoder and before the Mesh Processor. The critical property of this location is that

- the encoding path is identical to the baseline (channel-concat of all input frames, one Grid-to-Mesh pass),
- Mamba’s contribution is a *residual* with a zero-initialised output projection, so at step 0 the full model exactly equals the baseline.

Any measured improvement is therefore attributable to the learned memory and not to a different encoding.

2.3 MZ-lite variant used in the current pilot

In the pilot currently in production (`mz_r4_m3_i32_seg32_h16_fullnorm`), we go one step further and decouple the memory module entirely from GraphCast’s forward pass:

1. We run frozen GraphCast to get a per-grid-point Markov forecast $\hat{\mathbf{q}}_{t+1}^{\text{base}}$ for a batch of 32 consecutive one-step problems.
2. A small `MZResidualMamba` then reads $(\mathbf{q}_t, \mathbf{r}_t^{\text{prev}})$ at every grid point and produces a correction $\hat{\mathbf{r}}_{t+1}$.
3. The corrected forecast is $\hat{\mathbf{q}}_{t+1}^{\text{corrected}} = \hat{\mathbf{q}}_{t+1}^{\text{base}} + \hat{\mathbf{r}}_{t+1}$.

Per-grid-point SSM. `MZResidualMamba` reshapes the incoming $[T, B, \text{lat}, \text{lon}, 2F]$ tensor to $[B \cdot \text{lat} \cdot \text{lon}, T, 2F]$, passes it through a single Mamba-style selective SSM block shared across all grid points, and projects back to a F -channel residual with a zero-initialised head. There is no explicit spatial coupling inside this module — the memory kernel is spatially diagonal, which is a deliberate simplification.

Resolved feature vector. The memory currently operates on four synoptic-scale variables:

$$F = 1(\text{msl_pressure}) + 13(\text{geopotential}) + 13(u\text{-wind}) + 13(v\text{-wind}) = 40.$$

Input dimension to the Mamba block is $2F = 80$ (concatenation of the current state and the shifted previous residual); *output dimension* is $F = 40$ (the residual correction).

2.4 Anatomy of one Mamba block

Within the block, with hidden size H , the computation is (using $\odot$ for elementwise):

1. $\tilde{\mathbf{x}} = \text{LayerNorm}(\mathbf{x})$
2. $[\mathbf{u}; \mathbf{g}] = \text{Linear}_{2H}(\tilde{\mathbf{x}})$, split into value and gate.
3. $\Delta = \text{softplus}(\text{Linear}_H(\mathbf{u}))$, the data-dependent time step.
4. Learnable parameters $\mathbf{a}_{\log} \in \mathbb{R}^H$, $\mathbf{s} \in \mathbb{R}^H$ (init -0.1 and 1.0), with effective decay $a = -e^{\mathbf{a}_{\log}}$.
5. Recurrence over time via `jax.lax.scan`: for $t = 1, \dots, T$,

$$\text{decay}_t = \exp(a \odot \Delta_t), \quad h_t = \text{decay}_t \odot h_{t-1} + (1 - \text{decay}_t) \odot \mathbf{u}_t,$$

$$y_t = h_t \odot \sigma(\mathbf{g}_t) + \mathbf{s} \odot \mathbf{u}_t, \quad h_0 = 0.$$

6. Output projection $\text{Linear}_D(\mathbf{y})$, then $\mathbf{x}' = \mathbf{x} + \mathbf{y}$ (residual).

The final residual head (`Linear:  $H \rightarrow F$`) is initialised to *zero*, so at step 0 the predicted residual is identically zero.

3 Sampling: how a training example is built

3.1 Year layout and time continuity

The local dataset contains three years of ERA5 at 6-hour cadence. We train on 2020–2021 (adjacent) and hold out 2022 for both validation and final inference. This avoids the pitfall of concatenating non-adjacent years, which would inject a one-year discontinuity into forcings (year progress, solar radiation, accumulated precipitation) and cause the frozen baseline to produce NaN at the join.

A defensive filter in the training script drops any sample whose full input+target window is not strictly $\Delta t = 6$ h-consecutive, so misconfigured year selections fail loudly rather than silently poisoning the loss.

3.2 Sequential segments

Valid final-input indices in the training set are split into *contiguous segments* of length $S = 32$ using

$$\text{_time_continuous_segments}(ds, \text{indices}, S, \Delta t),$$

which is like `build\_sequential\_segments` in the main repo but splits on actual timestamp gaps, not just index gaps.

3.3 One training step

For each segment $\mathcal{S} = (i_1, \dots, i_{32})$:

1. Call `build_batch_from_indices` which returns an xarray batch with leading axis 32; each element is an independent one-step sample with a 32-frame history window (192 h).
2. Run frozen GraphCast once on the whole batch of 32 to obtain $\hat{\mathbf{q}}_{i_k+1}^{\text{base}}$ for $k = 1, \dots, 32$.
3. Extract the four resolved variables at the last input time ($\mathbf{q}_{i_k}$, “current state”), at the target time ($\mathbf{q}_{i_k+1}^{\text{truth}}$), and from the baseline prediction.
4. Reshape from $[\text{batch} = 32, \dots, F]$ to $[T = 32, B = 1, \text{lat}, \text{lon}, F]$: the 32 batch elements are *reinterpreted* as a $T = 32$ time sequence for the SSM. This is valid because segments are strictly time-sorted and never shuffled internally.
5. Build the teacher-forcing signal

$$\mathbf{r}_t^* = \mathbf{q}_t^{\text{truth}} - \hat{\mathbf{q}}_t^{\text{base}}, \quad \mathbf{r}_1^{\text{prev}} = 0, \quad \mathbf{r}_t^{\text{prev}} = \mathbf{r}_{t-1}^* \quad (t > 1).$$

6. Concatenate $[\mathbf{q}_t; \mathbf{r}_t^{\text{prev}}]$ into a length- $2F$ feature vector and feed the 32-step sequence into the Mamba block.
7. Compute a weighted MSE loss (§4) and take one AdamW step.

Training samples one segment per gradient step. Across segments they are shuffled; within a segment the 32-step order is preserved.

3.4 Teacher forcing and its caveat

During training and current evaluation, $\mathbf{r}_t^{\text{prev}}$ is the *true* previous residual. At deployment, this needs to be replaced by the model’s own $\hat{\mathbf{r}}_{t-1}$. The gap between teacher-forced and autoregressive metrics is the true error-accumulation rate and should be measured before claiming deployment skill.

4 Loss: GraphCast-style weighted MSE in normalised units

GraphCast internally computes its loss in normalised residual space (via the `InputsAndResiduals` wrapper). Since our MZ output bypasses that wrapper, we apply the same weighting scheme explicitly:

- **Per-channel scale:** divide each channel’s error by the one-step diff stddev `diffs_stddev_by_level`. This makes the loss dimensionless and balances variables of vastly different magnitudes (geopotential $\sim 10^5$ vs. wind ~ 10).
- **Latitude weight:** $\cos(\text{lat})/\cos(\text{lat})$, accounting for sphere-slice area.
- **Level weight:** $p/\bar{p}$ on the 13 task pressure levels, giving more weight to near-surface levels and less to the stratosphere.
- **Per-variable weight:** $w_{\text{msl}} = 0.1$, others = 1.0 (matching GraphCast’s default).

All weights are normalised to unit mean along their respective dimensions, so the final loss remains an $O(1)$ dimensionless quantity.

Input standardisation. Symmetrically, the inputs to the Mamba block are standardised:

- $\mathbf{q}_t$ is z -scored using `mean_by_level` and `stddev_by_level`.
- $\mathbf{r}_t^{\text{prev}}$ is divided by `diffs_stddev_by_level` (it has zero mean by construction).
- The network outputs a *normalised* residual, which is de-normalised by $\times \text{diffs_stddev}$ before being added to the baseline.

Without this, the linear layer $\text{Linear}(2F \rightarrow H)$ would be dominated by the pressure/geopotential directions and blow up within a handful of training steps.

Numerical hygiene.

- Frozen GraphCast runs in `fp32` here, not `bf16`. In `bf16` the quantisation noise on pressure (~ 780 Pa) is on the same scale as the one-step residual (~ 200 Pa) and contaminates the target signal.
- Global-norm gradient clipping at 1.0 is always on.
- Optional linear LR warmup over 200 steps (current: on).

5 Current parameters

5.1 Run configuration for the 2026-04-21 pilot

Group / parameter	Value
<i>Spatial grid</i>	
<code>resolution</code>	4° (46 latitudes $\times$ 90 longitudes; poles included)
<code>mesh_size</code>	3 (icosahedral refinement $\Rightarrow$ 642 mesh nodes)
<i>Time</i>	
<code>input_duration</code>	192h $\Rightarrow$ <code>input_steps</code> = 32 history frames
<code>target_steps</code> (baseline)	1
<code>segment_steps</code> (MZ)	32 (8 days of contiguous samples)
<code>train_years</code> / <code>val_year</code>	2020–2021 / 2022
<i>Frozen baseline (GraphCast)</i>	
<code>baseline_ckpt</code>	<code>bfix_r4_m3_in32_t1/ckpt_step23000.npz</code> (eval loss ~ 3.40)
<code>baseline_precision</code>	<code>fp32</code>
<i>MZ residual module (MZResidualMamba)</i>	
<code>hidden_size H</code>	16
<code>layers</code>	1
<code>dropout</code>	0.0
<code>a_log_init</code>	-0.1 (decay ≈ 0.9 /step $\Rightarrow$ multi-day memory)
<code>input_size</code>	$2F = 80$
<code>output_size</code>	$F = 40$ (<code>msl_press</code> + 13-level <code>z</code> , <code>u</code> , <code>v</code>)
Parameter count	$\sim 3\,100$ trainable weights
<i>Training</i>	
<code>max_steps</code>	400 (one gradient step per segment)
<code>eval_every</code> / <code>eval_max_segments</code>	100 / 8
<code>checkpoint_every</code>	200
<code>lr</code>	10^{-4} (linearly warmed from 0 over 200 steps)
<code>weight_decay</code>	10^{-4}
<code>grad_clip</code>	1.0 (global norm)
<code>seed</code>	0
<code>precision</code> (compute)	<code>bf16</code> hint; MZ network actually runs <code>fp32</code> end-to-end
<i>Normalisation (all on by default)</i>	
<code>standardize_input</code>	<code>z-score current_state</code> ; divide <code>prev_residual</code> by diffs std
<code>normalize_loss</code>	weighted MSE in diffs-std space
Latitude weight	$\cos(\text{lat})/\overline{\cos(\text{lat})}$
Level weight	$p/\bar{p}$ on 13 task levels
Per-variable weight	<code>msl_press</code> = 0.1, others = 1.0

5.2 Resolution sweep in progress

To separate the contribution of resolution from everything else, we are now running four apples-to-apples configurations at `res` $\in \{2, 4, 6, 8\}^\circ$ with all other settings held fixed: `mesh_size`=3,

width=128, input_duration=192h, segment_steps=32, hidden_size=16, shared random seed. Each MZ run will be trained on top of its own baseline’s ckpt_step5000.npz so all four MZ runs see a baseline of comparable strength.

6 Representative results (res=4, step 400)

After 400 gradient steps the MZ module is clearly learning and reduces per-variable MAE over the frozen baseline on the 2022 validation set:

Variable	Baseline MAE (frozen)	MZ-corrected MAE (step 400)	Improvement
mean_sea_level_pressure	86.6 Pa	80.8 Pa	−6.78%
geopotential	81.6 m ² /s ²	74.8 m ² /s ²	−8.28%
u_component_of_wind	2.10 m/s	2.08 m/s	−1.07%
v_component_of_wind	2.14 m/s	2.13 m/s	−0.81%
OVERALL (weighted mix)	30.06	27.70	−7.84%

The gain is concentrated in large-scale, long-persistence fields (`geopotential`, `mean_sea_level_pressure`), which is the regime where temporal memory is expected to help most. Chaotic small-scale variables (`u`, `v` wind) see only marginal benefit. The curves have not converged at step 400, so further training is expected to extend the gap.

7 Known limitations

1. **Teacher-forced evaluation.** Eval uses true previous residuals; autoregressive eval (feed the model’s own $\hat{r}$ back in) is not yet wired up. The gap between the two will bound the true deployment skill.
2. **Spatially diagonal memory.** The Mamba SSM runs per grid point with no direct spatial coupling. A true MZ kernel can be non-local; a small GNN layer inside the memory module could relax this.
3. **Four resolved variables only.** Temperature, specific humidity, vertical velocity, and precipitation are not corrected. Extend once the synoptic variables show clear wins.
4. **Segment-boundary state reset.** The SSM state $h_0 = 0$ at the start of every 32-step segment. Real deployment has a continuous history; cross-segment state threading (as already done in the GraphCast-inline stateful Mamba) is a plausible extension.
5. **Short training schedule.** The current pilot uses `max_steps=400`; eval curves are still falling. A longer schedule is cheap (each step is ~ 14 s at 4°).
6. **Single baseline snapshot.** The quality of the frozen baseline strongly affects how much residual memory there is left to capture. We plan to compare MZ on top of weak (`step5000`) vs. strong (`step23000`) baselines.

8 Planned fix: from teacher forcing to autoregressive residual rollout

The main train–test mismatch in the current pilot is that both training and the current evaluation use the *true* previous residual as input,

$$r_1^{\text{prev}} = 0, \quad r_t^{\text{prev}} = r_{t-1}^*, \quad r_{t-1}^* = q_{t-1}^{\text{truth}} - q_{t-1}^{\text{base}}, \quad (1)$$

whereas at deployment this quantity is unavailable. In practice, the deployed model must use its *own* previously predicted residual,

$$r_1^{\text{prev}} = 0, \quad r_t^{\text{prev}} = \hat{r}_{t-1}. \quad (2)$$

The teacher-forced metrics reported above are therefore optimistic: they do not yet measure the true error-accumulation rate of the residual-memory module.

Step 1: add autoregressive evaluation. Before changing training, we add a genuine autoregressive evaluation mode. For a segment of length T , run

$$\hat{r}_t = f_\theta(q_t, r_t^{\text{prev}}, h_t), \quad (3)$$

$$q_{t+1}^{\text{corr}} = q_{t+1}^{\text{base}} + \hat{r}_{t+1}, \quad (4)$$

$$r_{t+1}^{\text{prev}} = \hat{r}_t, \quad (5)$$

and compare against the current teacher-forced evaluation. This directly quantifies the gap between ideal-context performance and true deployed performance.

Step 2: reduce exposure bias during training. If the autoregressive gap is significant, training should switch from pure teacher forcing to a mixed strategy such as scheduled sampling:

$$r_t^{\text{prev}} = \begin{cases} r_{t-1}^*, & \text{with probability } p, \\ \hat{r}_{t-1}, & \text{with probability } 1 - p. \end{cases} \quad (6)$$

The teacher-forcing probability p should start near 1 early in training and be gradually reduced toward 0. A practical implementation is a two-stage schedule: (i) warm start with pure teacher forcing, then (ii) continue training with mixed teacher/model residual feedback.

Step 3: train on rollout loss rather than ideal-context correction. The loss should be computed on the corrected autoregressive rollout,

$$\mathcal{L}_{\text{rollout}} = \sum_{t=1}^T w_t \ell(q_t^{\text{corr}}, q_t^{\text{truth}}), \quad (7)$$

so that the model is explicitly optimised for stability under self-generated residual feedback.

Step 4: add simple stabilisers if needed. If autoregressive rollout is unstable, two cheap stabilisers should be tested:

1. residual clipping, $\hat{r}_t \leftarrow \text{clip}(\hat{r}_t, -c, c)$, to suppress rare exploding corrections;
2. residual shrinkage, $q_{t+1}^{\text{corr}} = q_{t+1}^{\text{base}} + \beta \hat{r}_{t+1}$ with $0 < \beta \leq 1$, to damp over-aggressive corrections at inference time.

Recommended order of implementation. The modifications should be introduced in the following order:

1. add autoregressive evaluation without changing training;
2. measure the teacher-forced vs. autoregressive gap by variable and rollout step;
3. if the gap is large, introduce scheduled sampling;
4. if needed, add clipping or shrinkage as inference stabilisers.

Interpretation. This change does not alter the basic MZ-lite decomposition

$$\text{forecast} = \text{frozen Markov baseline} + \text{learned residual memory},$$

but makes the residual-memory module operate under the same information constraints at training, evaluation, and deployment. The goal is to turn the current residual corrector from an ideal-context predictor into a genuinely autoregressive memory model.

9 File layout for this pilot

Path	Role
src/models/mz_residual_mamba.py	MZResidualMamba, _SelectiveSSMBlock, shift_residual_history.
scripts/training/train_mz_residual_memory.py	Training loop: load frozen baseline, build segments, compute residual targets, optimise MZ params only.
scripts/training/infer_mz_save_tensors.py	Inference-only driver; saves baseline , corrected , truth tensors for plotting.
slurm_pilots/mz_r{2,4,6,8}_m3_i32_seg32_816fix11noscripts.sh	Slurm script for the four-resolution sweep.
slurm_pilots/bfix_r{2,4,6,8}_m3_in32_t1.Baseline	Baselines for the same sweep.
Weather_Global_experiments/results/mz_r{2,4,6,8}_m3_i32_seg32_t16_fix11noscripts.sh	Slurm script for the four-resolution sweep, checkpoints, plots.